

# An End-to-End AI Agent for Kaggle Competitions

Benchmarked against two frozen reference agents on 23 Kaggle competitions

Wei-Hao Huang

## Abstract

We present the **Sinica Kaggle Agent**, an end-to-end agent that runs a machine-learning competition from raw data to submitted prediction with no human in the loop. A large language model (LLM) reasons and decides at every stage while AutoML libraries are invoked through the shell, and a tree search over complete candidate solutions serves as the optimization loop. The agent was run in isolated sandboxes for all 23 competitions of the study (transcript-audited on the 20 ordinary ones) and benchmarked against two frozen, methodologically distinct reference agents, **AIDE** (LLM tree search) and the **NVIDIA Kaggle Agent** (public-kernel reproduction). The 23 competitions comprise 20 ordinary ones, whose verdicts come from real submissions scored on the live leaderboard because an agent’s own cross-validation cannot arbitrate between agents, and 3 special ones spanning language, physiological time series, and medical imaging, graded offline with MLE-bench. **Among the 23 competitions, Sinica obtains the best performance, with 13 wins and 10 losses**, taking the best private score on 10 of the 20 ordinary competitions and the three-way best on all 3 special ones.

## 1 Introduction

A Kaggle competition scores every submission on a hidden test set and ranks thousands of teams by that number. Solving one has meant expert labor and trial and error. But the score gives a machine a target, and the work becomes a loop that a machine can optimize. LLM-driven coding agents have recently begun to automate this loop. AIDE performs LLM-guided tree search over candidate solutions [1]; the system of Aygün et al. couples an LLM rewriter with tree search and externally injected research ideas, outperforming strong baselines across 16 Kaggle Playground competitions [2]; and industrial agents skip search altogether by reproducing the strongest public solution to a given competition [3].

This report is about one such agent, **the Sinica Kaggle Agent**, a hybrid design in which an LLM reasons at every stage of the competition pipeline, AutoML libraries are invoked through the shell, a cross-competition experience library supplies evidence-backed priors, and a tree search over candidate solutions serves as the optimization loop (Figures 1 and 2). It also auto-documents. Each run yields a per-competition specification spanning data, models, training, inference, and evaluation.

To evaluate the Sinica agent, we benchmark it against two frozen reference agents, **AIDE** (LLM tree search with no cross-competition memory) and the **NVIDIA Kaggle Agent**, which reproduces the highest-voted public kernel. Sinica’s lanes ran in isolated sandboxes across all 23 competitions; on the 20 ordinary ones each lane’s transcript was audited (Section 3) and the final submission was scored on the real leaderboard via late submission, while the 3 special competitions are graded offline with MLE-bench [4]. Sinica takes the best score on 13 of the 23 competitions, the strongest standing of the three agents (Section 4).

## 2 Methods

### 2.1 The Agent

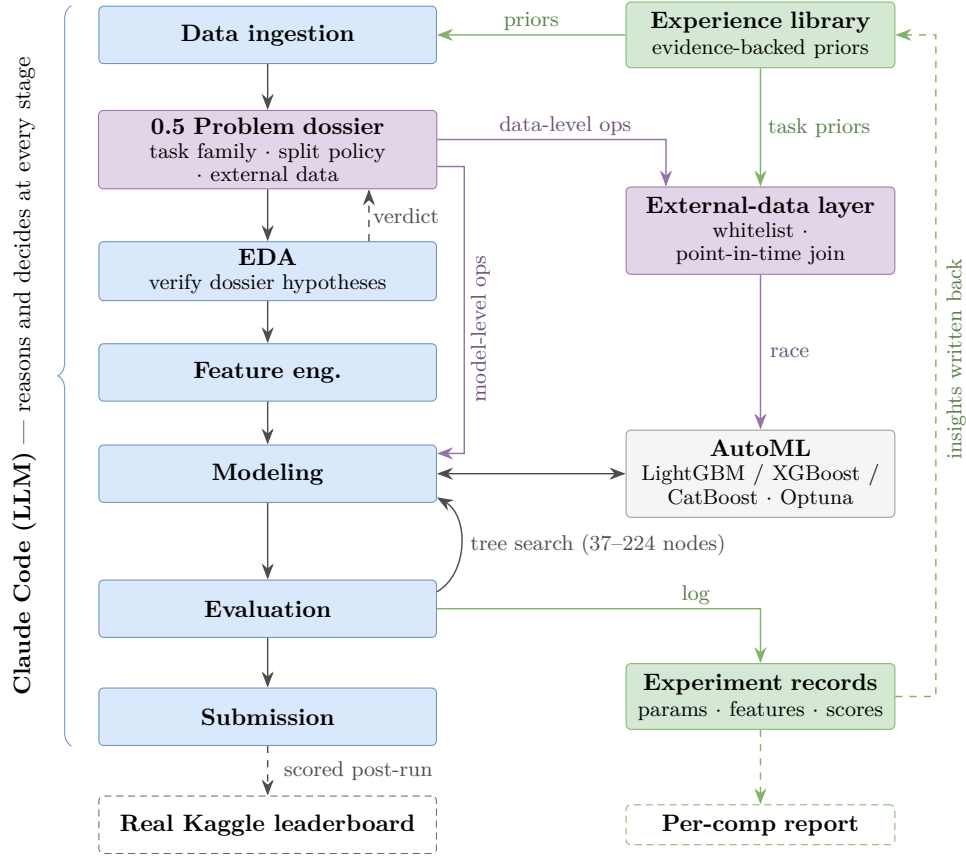
The method is implemented as a Claude Code Skill (`kaggle-agent`) around a division of labor. Execution follows the pipeline of Figure 1. The LLM (Claude Code) handles problem understanding, validation-strategy design, feature engineering, result interpretation, and the choice of next step, while LightGBM / XGBoost / CatBoost / Optuna, invoked via the shell, handle model selection, hyperparameter optimization, and stacking/ensembling.

The injection layer, the purple elements of Figures 1 and 2, supplies external knowledge at the head of the pipeline. An earlier downstream variant injected external ideas at the blend stage and measured a gain of only  $\sim 10^{-5}$  on the blend’s competition metric; the architecture moves the injection point upstream to problem identification through two components:

- **Stage 0.5, the problem dossier:** before EDA, the agent answers “what kind of problem is this?” by naming task family, train–test window relation, split policy, and external-data need in `dossier.json`, grounded in a structured library of evidence-backed task-level priors (`knowledge/knowledge_base.json`), rendered per competition by `knowledge/task_priors_for.py`. Reading competition-specific discussions or kernels is forbidden; this keeps the method distinct from reproduction-based agents. The dossier is a prior, not a conclusion; EDA must verify its hypotheses and record a verdict.
- **External-data module:** whitelisted public sources are joined through leakage-guarded as-of merges with a declared lag. The module was adversarially audited, and the surfaced flaws, including silent-future-leak classes, were fixed under regression tests. The dossier and the module share one closed vocabulary of typed operators, so an unrealizable proposal leaves a trace in a coverage ledger rather than a silent success. On the three *firing-class* competitions where the layer proposes external data and a public/private leaderboard exists to adjudicate (s3e19, tps-sep-2022, s5e1), all six external-data arms beat their matched no-external baselines on the private leaderboard, and the s3e19 ablation moves the private score from 48.243 to 50.455 (full arm scores in the Appendix).

Two record-keeping mechanisms support the pipeline (Figure 1):

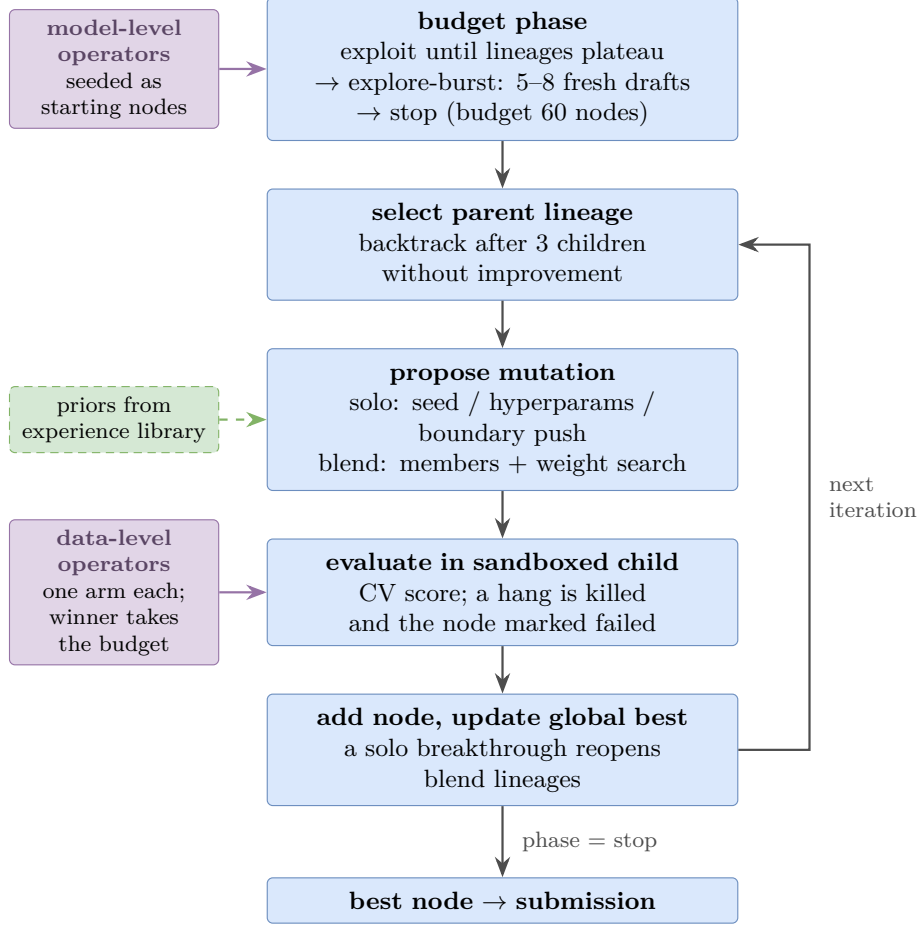
- **Experiment records:** every experiment’s parameters, features, split scheme, and scores are logged in structured form (`experiments.json`, `experiments_tree_v3.json`), and each lane records a structured exploratory-data-analysis (EDA) pass (distributions, drift, correlation) before modeling, which the skill’s reporting layer renders, with the dossier and experiment records, into a per-competition ML specification report (see the Appendix).
- **Cross-competition experience library** (`knowledge/experience.md`): every insight must carry an evidence field (`competition`, `exp #N`, `score A`  $\rightarrow$  `score B`); entries without a score delta are rejected. The agent queries the library before EDA and before modeling, and writes validated new insights back after experiments. This is the main design difference between Sinica and the two reference methods.



**Figure 1:** Architecture of the Sinica Kaggle Agent, with color marking roles. **Blue, the LLM-driven pipeline:** Claude Code reasons and decides at every stage. **Purple, the injection layer:** the dossier’s typed operators enter as external-data arms or search lineages. **Green, the evidence loop:** the experience library and the experiment records. **Gray, AutoML tools** invoked via the shell. Dashed arrows mark feedback, advisory, or post-run steps.

Modeling and evaluation run in one of two optimization modes:

- **Linear iteration:** propose a hypothesis, run the experiment, record the result (used for the first pass and for small competitions).
- **Tree search:** once linear iteration has produced a single baseline model and at least one blend, the agent switches to tree search. Every ordinary-competition run in this report uses the tree-search harness of Figure 2 (`tree_search/harness_v3.py`).



**Figure 2:** The tree-search loop used in all Sinica runs (default budget 60 evaluated nodes per competition; per-lane counts are in the experiment logs). Each node is a complete, executable solution scored by local cross-validation; blends over cached out-of-fold (OOF) predictions are first-class nodes, and the best node becomes the submission. Purple: the injection layer’s operator entry points; green: priors from the experience library; dashed arrows: advisory input and backtracking.

## 2.2 The Two Reference Agents

**AIDE** (first-generation open-source release) performs LLM-driven tree search. At each step, it writes a complete solution, executes it, reads the error or score, and mutates accordingly. It carries no cross-competition memory.

**NVIDIA Kaggle Agent** (skill) researches the competition context, selects the highest-*voted* solution-type kernel, ports it faithfully, and retrains on local data, reproducing the strongest public kernel.

Both reference methods are frozen, with no improvements, even where we found weaknesses (the NVIDIA skill ships a score-based selection tool we deliberately left disabled). They stay frozen because improving a reference agent mid-study would make earlier and later data incomparable and would quietly turn the benchmark into an optimization of the reference methods.

## 3 Benchmark Design

**Evaluation set.** The study uses 23 competitions in two groups, mirroring the two scoring routes available. The 20 *ordinary* competitions are tabular Kaggle competitions with live late submission, comprising 7 classification tasks (binary and ordinal) and 13 prediction/regression tasks including time series and multi-target, with training sets from 1,157 to 630,000 rows (Table 1). The 3 *special* competitions sit outside

the tabular family. They are us-patent (NLP text pairing), ventilator-pressure-prediction (physiological time series), and SIIM-ISIC melanoma (dermoscopic imaging), prepared and graded offline by MLE-bench [4], which replays finished Kaggle competitions with their original data and metrics.

**Table 1:** Competition descriptions.

| #  | Competition                    | Task type                            | Train rows $\times$ cols |
|----|--------------------------------|--------------------------------------|--------------------------|
| 1  | s3e1                           | Regression                           | $37,137 \times 10$       |
| 2  | s3e3                           | Classification (binary)              | $1,677 \times 35$        |
| 3  | s3e5                           | Classification (ordinal)             | $2,056 \times 13$        |
| 4  | s3e7                           | Classification (binary)              | $42,100 \times 19$       |
| 5  | s3e9                           | Regression                           | $5,407 \times 10$        |
| 6  | s3e11                          | Regression                           | $360,336 \times 17$      |
| 7  | s3e14                          | Regression                           | $15,289 \times 18$       |
| 8  | s3e16                          | Regression                           | $74,051 \times 10$       |
| 9  | s3e19                          | Regression (time series)             | $136,950 \times 6$       |
| 10 | s4e11                          | Classification (binary)              | $140,700 \times 20$      |
| 11 | s5e10                          | Regression                           | $517,754 \times 14$      |
| 12 | s6e1                           | Regression                           | $630,000 \times 13$      |
| 13 | s6e2                           | Classification (binary)              | $630,000 \times 15$      |
| 14 | afsis-soil-properties          | Regression (multi-target)            | $1,157 \times 3,600$     |
| 15 | cat-in-the-dat                 | Classification (binary)              | $300,000 \times 25$      |
| 16 | conway-s-reverse-game-of-life  | Structured multi-output              | $50,000 \times 802$      |
| 17 | tps-aug-2022                   | Classification (binary)              | $26,570 \times 26$       |
| 18 | tps-jan-2022                   | Regression (time series)             | $26,298 \times 6$        |
| 19 | s5e1                           | Regression (time series)             | $230,130 \times 6$       |
| 20 | tps-sep-2022                   | Regression (time series)             | $70,128 \times 6$        |
| 21 | us-patent-phrase-matching      | Regression (text similarity)         | $32,825 \times 5$        |
| 22 | ventilator-pressure-prediction | Regression (time series, seq-to-seq) | $5,432,400 \times 8$     |
| 23 | siim-isic-melanoma             | Classification (binary, imaging)     | $28,984 \times 8$        |

Rows 1–20: the ordinary competitions (7 classification, 13 prediction/regression). Rows 21–23: the 3 special competitions; sizes are the MLE-bench re-split training sets.

**Execution conditions.** The three agents ran serially on one machine, each method as shipped. AIDE ran its default 20 steps, NVIDIA reproduced public kernels per its method, and Sinica ran the full six-stage pipeline including tree search (default budget 60 evaluated nodes per competition, extendable through the logged resume path). Four isolation conditions (data, workspace, instruction, and resource) are explicitly enforced (enforcement details in the Appendix).

**Sandboxes, audit, and scoring.** Each of Sinica’s 20 ordinary-competition lanes ran in an isolated root with the Kaggle token stripped, so the agent could neither consult the leaderboard nor submit. After a lane finished, an audit read every tool call in its transcript before the score was accepted. An accepted lane produces a final `submission.csv`, which we submit from outside the sandbox to the finished competition for the official public and private scores.

## 4 Results

### 4.1 The 20 Ordinary Competitions

Table 2 gives the complete benchmark, the public and private leaderboard scores of all three agents on the 20 competitions. **Sinica takes the best private score on 10 of the 20** (NVIDIA 7, AIDE 3).

**Table 2:** The complete 20-competition benchmark.

| #                 | Competition    | Metric     | Sinica LB                       | NVIDIA LB                          | AIDE LB                        |
|-------------------|----------------|------------|---------------------------------|------------------------------------|--------------------------------|
| 1                 | s3e1           | RMSE ↓     | <b>0.55275</b> /0.55715 (77.1)  | 0.55550/0.55332 (93.2)             | 0.55552/0.55899 (68.0)         |
| 2                 | s3e3           | ROC-AUC ↑  | 0.87157/0.89449 (30.6)          | <b>0.89537</b> /0.93526 (66.8)     | 0.86863/0.88484 (26.4)         |
| 3                 | s3e5           | QWK ↑      | 0.57223/0.57041 (72.0)          | 0.56974/0.57515 (75.7)             | <b>0.58138</b> /0.59362 (84.4) |
| 4                 | s3e7           | ROC-AUC ↑  | 0.90441/0.91127 (74.3)          | <b>0.90458</b> /0.91230 (78.1)     | 0.90109/0.90934 (67.1)         |
| 5                 | s3e9           | RMSE ↓     | 12.33192/11.84750 (51.2)        | <b>12.22913</b> /11.83399 (59.1)   | 12.30667/11.86657 (44.5)       |
| 6                 | s3e11          | RMSLE ↓    | <b>0.29368</b> /0.29309 (76.6)  | 0.29624/0.29559 (66.6)             | 0.29445/0.29389 (73.3)         |
| 7                 | s3e14          | MAE ↓      | 333.18611/343.43434 (66.3)      | <b>330.71616</b> /338.36524 (89.2) | 332.31556/342.44918 (71.2)     |
| 8                 | s3e16          | MAE ↓      | <b>1.33822</b> /1.33738 (95.1)  | 1.34001/1.33566 (96.4)             | 1.34224/1.33920 (91.4)         |
| 9                 | s3e19          | SMAPE ↓    | 49.40878/49.11872 (37.6)        | <b>48.26558</b> /47.52145 (41.4)   | 48.34131/48.39715 (39.9)       |
| 10                | s4e11          | Accuracy ↑ | <b>0.94107</b> /0.94195 (64.0)  | 0.94076/0.94136 (52.8)             | 0.94056/0.94184 (62.1)         |
| 11                | s5e10          | RMSE ↓     | <b>0.05576</b> /0.05554 (81.2)  | 0.05588/0.05562 (59.4)             | 0.05579/0.05555 (78.9)         |
| 12                | s6e1           | Error ↓    | <b>8.71516</b> /8.68844 (76.8)  | 8.73239/8.70480 (69.5)             | 8.72191/8.69449 (74.8)         |
| 13                | s6e2           | ROC-AUC ↑  | 0.95505/0.95354 (68.6)          | 0.95508/0.95348 (63.0)             | <b>0.95516</b> /0.95364 (76.5) |
| 14                | afsis          | MCRMSE ↓   | <b>0.48917</b> /0.41902 (72.7)  | 0.69855/0.61276 (18.8)             | 0.49861/0.45636 (45.6)         |
| 15                | cat-in-the-dat | ROC-AUC ↑  | <b>0.80243</b> /0.80800 (72.6)  | 0.77084/0.77641 (32.6)             | 0.80217/0.80790 (70.3)         |
| 16                | conway         | MAE ↓      | <b>0.10414</b> /0.10313 (100.0) | 0.11189/0.11094 (96.5)             | 0.11065/0.10960 (97.9)         |
| 17                | tps-aug-2022   | ROC-AUC ↑  | 0.58930/0.58534 (63.5)          | 0.58730/0.57920 (43.0)             | <b>0.59106</b> /0.58479 (60.5) |
| 18                | tps-jan-2022   | SMAPE ↓    | 6.29895/4.58859 (75.4)          | <b>4.63651</b> /4.43958 (76.4)     | 10.66638/10.11598 (28.9)       |
| 19                | s5e1           | MAPE ↓     | <b>0.08401</b> /0.09076 (74.3)  | 0.14594/0.05726 (84.4)             | 0.17917/0.12573 (61.6)         |
| 20                | tps-sep-2022   | SMAPE ↓    | 28.49605/28.45070 (13.6)        | <b>5.48351</b> /4.77458 (81.1)     | 9.72095/5.78998 (72.1)         |
| Median PR (range) |                |            | 72.7 (13.6–100.0)               | 68.2 (18.8–96.5)                   | 69.2 (26.4–97.9)               |

Cells: private/public (PR) LB scores; **bold** = best *private* (direction-aware); PR = percentile rank of the public score against 2026-08-24 public-leaderboard snapshots.

### 4.2 The 3 Special Competitions

The three agents also ran the three special competitions. Because one node is a full deep-learning fine-tune rather than a gradient-boosting refit that takes seconds, a tree search over dozens of nodes would overrun the lane’s compute budget. Therefore, on each of the three competitions, Sinica only ran the linear-iteration mode of Section 2.1. It iterated to one validated model, then built and submitted a blend of the models it had trained. MLE-bench then grades each submission offline on its held-out private split. Sinica takes the three-way best on all 3 special competitions (Table 3). Winning across three unrelated task families suggests the advantage comes from pipeline discipline as well as from tree search and external-idea injection.

**Table 3:** The 3 special competitions, graded by MLE-bench.

| Competition | Metric        | Sinica                    | NVIDIA            | AIDE    |
|-------------|---------------|---------------------------|-------------------|---------|
| us-patent   | Pearson $r$ ↑ | <b>0.86658</b> † (silver) | 0.86160† (bronze) | 0.65821 |
| ventilator  | MAE ↓         | <b>0.16525</b>            | 0.40093           | 0.34591 |
| SIIM-ISIC   | AUC ↑         | <b>0.93529</b> †          | 0.89822           | 0.88579 |

**Bold** = best of the three; † = above the frozen leaderboard’s median.

### 4.3 Final Standing Across 23 Competitions

Counting the best score on each of the 23 competitions, **Sinica performs best, winning 13 and losing 10**. The other 10 wins, 7 by NVIDIA and 3 by AIDE, all come on ordinary competitions (Table 2).

Both of Sinica’s heaviest losses occur where a mature public solution exists. On tps-sep-2022, the public baseline is so strong that both reference agents beat Sinica by a wide margin (SMAPE 5.48 and 9.72 against 28.50); on tps-jan-2022, NVIDIA reproduced the well-known GDP-ratio solution and won 4.64 to 6.30. Reproduction wins where there is a strong solution to reproduce; the search-based agents take most of the rest.

## 5 Discussion

### 5.1 Comparing the Three Agents

The three agents differ in where knowledge comes from and in how the next step is decided rather than in tuning skill (Table 4).

**Table 4:** Design comparison of the three methods.

| Aspect              | Sinica                                                           | AIDE                                  | NVIDIA                                 |
|---------------------|------------------------------------------------------------------|---------------------------------------|----------------------------------------|
| Knowledge source    | Cross-competition experience library (accumulating)              | None (from zero each time)            | Public kernels fetched fresh each time |
| Search strategy     | Six-stage pipeline + tree search (60-node default budget)        | Tree-shaped trial and error, 20 steps | No search; faithful reproduction       |
| Basis for next step | Experience-library priors + OOF diagnostics                      | Previous step’s error or score        | Highest-voted kernel                   |
| Validation rigor    | Split chosen by task type; diagnostics tagged and excluded       | No concept of split legality          | Inherits the kernel’s split            |
| External data       | Injection layer (typed operators, whitelisted sources)           | Never considered                      | Inherited from the kernel              |
| Run documentation   | Writes its own five-section specification report per competition | None                                  | None                                   |

The reference agents’ wins track their knowledge source. Sinica’s edge is validation rigor and accumulated knowledge (on one competition it quarantined the same shuffled split that AIDE submitted; see the companion case studies), while its measured bottleneck was judgment at problem identification, the gap the injection layer was built to close.

### 5.2 Disadvantages of the Agent-Based Approach

Running one agent through 23 competitions and reading every transcript also documents where the approach falls short.

- **The agent is strong at coding, weak at judging.** The agent efficiently works through whatever problem it is given; it does not ask whether a change is worth making. This often leads to a poor design choice that yields bugs, and the cost surfaces only after results return, when the choice is expensive to unwind (the downstream injection variant of Section 2.1 is our own instance).
- **The agent can only target the local validation score.** Inside a lane, the leaderboard is unreachable by construction, so iteration optimizes cross-validation (CV) alone. As a result, the local validation score can look outstanding while the real leaderboard score is mediocre. The extreme case is s3e19’s pooled OOF SMAPE of 6.71 against its realized 49.41, and it is why no local CV number is quoted as evidence anywhere in this report.

- **The agent often buries the main question in detail.** AI-written reports put too much focus on technical details and drift away from what the reader wants to know; every specification report attached to this study needed human editing for exactly this failure.
- **The agent has bad writing habits.** Redundant headings and reinvented terms come back draft after draft, and verifying and modifying them by hand is exhausting; unlike a score, prose quality has no automatic gate.

### 5.3 Practical Advice for Building AI Agents for Data Science

We offer three pieces of advice, each paid for by a measurement in this study.

- **Build an isolated environment before running the agent.** This prevents any access to the leaderboard or another run’s files. Isolation must be structural (credentials stripped, roots separated, library mounted read-only), not behavioral. The one cross-lane contamination in this study came from a writable shared file, not from an agent breaking a rule it was given (Section 3).
- **Justify the design before building the pipeline.** This reduces redesign cost and debugging time and avoids a full rerun of earlier competitions. An architecture change invalidates every lane run before it, and this benchmark archived seven partly finished runs for exactly that reason.
- **Inject external knowledge before data modeling.** This often improves the score; if the external data are injected at the final step instead, the score shows almost no improvement ( $\sim 10^{-5}$  in our downstream variant, against six-for-six controlled wins for the upstream layer, Section 2.1).

### 5.4 Limitations

- **The sample is small and the standing is sensitive to its composition.** Reproduction-based and search-based methods win on disjoint kinds of competitions, so adding or removing a few competitions of one kind moves the win count. Reproduced kernels were also tuned by their authors against the realized public leaderboard, so the comparison is least symmetric exactly where the reproduction method is strongest.
- **The special-competition lanes ran the documented fallback, not the full method** (Section 4.2), and offline grading yields one score per submission, with no public/private pair to corroborate it.

### 5.5 Open Problems

The sharpest open question is operator-form selection. None of the three local signals we tested selects the form the injection should take, and one selector was killed by its own pre-registered test on s5e1. A fourth candidate, horizon length, emerged only after the fact; we have committed it as a pre-registration, to be tested on the next firing-class competition before its leaderboard is consulted. Second, few competitions need external data at all, so the controlled evidence rests on three firing-class competitions, only two of which (tps-sep-2022 and s5e1) played no part in building the task-prior library the dossier consults; extending that set is worth more than re-running competitions where the layer stays silent. Third, the experience library’s residual weak link is retrieval quality. Enforcement is closed (an experiment record without a library-query trace is invalid, checked mechanically), but no mechanism yet scores retrieved priors for applicability, and a direct ablation of the library remains to be run.

## 6 Conclusion

All 23 competitions ran in isolated sandboxes, the 20 ordinary ones transcript-audited and scored on the real leaderboard, the 3 special ones graded offline by MLE-bench. Among the 23 competitions, Sinica

performs best, with 13 wins and 10 losses, taking the best private score on 10 of 20 and the three-way best on all 3 special competitions. Its losses concentrate where a mature public solution exists and reproduction beats search. Two negative results accompany the study. No local signal selects the injection's form, and the audit discarded a contaminated lane whose tainted score looked better than its clean replacement's. This study aimed to build a benchmark that can kill its own hypotheses.

## Appendix: Experimental Procedure and Availability

**Fairness protocol.** Because the three agents share one machine, four isolation conditions are explicitly enforced. Two are enforced by tooling, one by directory structure, and one, instruction isolation, by a written protocol with an incident log (Table 5).

**Table 5:** The four isolation conditions and what enforces each.

| Condition   | Rule                                                                                                                                         | Enforcement                                          |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| Data        | Each side reads only its own root, containing files validated against the official Kaggle manifest                                           | <code>benchmark_infra/build_isolated_roots.py</code> |
| Workspace   | The three output directories are fully separated; each side is scored independently                                                          | Directory structure                                  |
| Instruction | Prompts may carry environment facts and metric thresholds only, never modeling recipes, another side’s scores, or a workaround it discovered | Manual protocol + incident log                       |
| Resource    | Only one lane executes at any moment                                                                                                         | <code>benchmark_infra/lane_lock.sh</code><br>mutex   |

**Per-competition specification reports.** A specification report accompanies this paper for each of the 23 Sinica runs (the 20 ordinary competitions, plus the 3 special competitions via a deep-learning-aware collector whose DL fields (backbone, learning-rate schedule, batch size, augmentation, transfer learning) are filled natively). Each was written by the agent at the end of the run it describes; neither reference agent documents its own runs, so no counterpart set exists (our collector re-documented AIDE’s runs after the fact, archived in the repository). All are written to the five sections of Tso-Jung Yen’s ML specification framework (Data / Models & Architecture / Training / Inference / Evaluation & Benchmarking), archived in `docs/ml_specs/` in Markdown and PDF. Every number in them is extracted deterministically by `.claude/skills/kaggle-mlspec-report/assets/collect.py` (tabular lanes), `benchmark_infra/collect_dl_facts.py` (deep-learning lanes), and `benchmark_infra/collect_aide_facts.py` from structured records, never generated by an LLM; unavailable fields are marked “not recorded.” The reports are CV-only by construction; their conclusions must not be mixed with leaderboard verdicts.

**External-data arm scores.** Table 6 restores the matched-configuration arm scores behind the six-for-six claim of Section 2.1. On `s5e1`, which joined the firing class later and ran searched arms only, the private MAPE scores are 0.15751 (baseline), 0.15626 (`join_feature`), and 0.12417 (`ratio_target`). All six external-data arms beat their matched baselines under these lower-is-better metrics.

**Table 6:** Matched-configuration arms: identical pipeline, no search, no blending, so the operator is the only difference. SMAPE, lower is better. Every external arm beats its own baseline on both leaderboards.

| Arm                       | s3e19        |               |               | tps-sep-2022 (held out) |               |               |
|---------------------------|--------------|---------------|---------------|-------------------------|---------------|---------------|
|                           | CV           | public        | private       | CV                      | public        | private       |
| no external data          | 10.148       | 52.492        | 54.506        | <b>11.348</b>           | 23.828        | 25.476        |
| <code>join_feature</code> | 9.379        | <b>47.036</b> | <b>48.497</b> | 11.624                  | <b>21.973</b> | <b>23.390</b> |
| <code>ratio_target</code> | <b>7.793</b> | 50.853        | 52.073        | 11.807                  | 21.209        | 24.091        |

**Execution environment.** All lanes ran on ARM64 Linux (Ubuntu 24.04) with an NVIDIA GB10 GPU, under a uv-managed Python 3.13 environment frozen in `uv.lock`, with fixed seeds and LightGBM set to `deterministic=true`, `force_row_wise=true`, and a fixed `num_threads`; without these settings, the same configuration does not reproduce across processes.

**Companion documents and availability.** Two companion documents carry the depth this report condenses. They are *Building the Injection Layer: An Engineering Log* (per-regime coverage tables, audit scripts, and the provenance of the two special-competition scores that had to be re-run) and *Per-Competition Case Studies* (the CV-leaderboard reversal mechanics, AIDE’s split switch, the s5e1 pre-registration setting, and the s3e19 residual decomposition). All code and benchmark artifacts are archived in [github.com/930731haohao-afk/kaggle-agent](https://github.com/930731haohao-afk/kaggle-agent) (private; access granted on request); every number in this report traces to those structured records.

## References

- [1] Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., and Wu, Y. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138 (2025).
- [2] Aygün, E., et al. An AI system to help scientists write expert-level empirical software. *Nature* **654**, 909 (2026). <https://doi.org/10.1038/s41586-026-10658-6>
- [3] NVIDIA. Kaggle Agent: kernel-reproduction agent skill (software), 2026. <https://github.com/NVIDIA/nvidia-kaggle>
- [4] Chan, J. S., et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. In *Proceedings of the 13th International Conference on Learning Representations*, 2025. <https://openreview.net/forum?id=6s5uXNWGIh>